EGC–301P: Project Report

cmux – Code Multiplexer

Anish Teja Bramhajosyula – IMT2024029

Problem Statement

cmux or **Code Multiplexer** is a collaborative code–editor which works live in real–time. Users can log in with accounts (approved by an administrator) and work on multiple files together. Users can create and share files to other users. Edits are tracked in real–time as they are typed by users, and synchronised to every other user currently accessing the file being edited. The line on which a user is typing is locked, so that no other user can trigger race conditions and this is shown clearly in the editor windows of the other users. Users can also manage their accounts by accessing their credentials and changing their passwords. The administrator can create/remove users, manage their accounts and see all the files in the remote repository. This entire foreground process is supported by a concurrent server dæmon running and logging client activity.

Implementation

1. ### Role–Based Authorisation

   There are two roles: Admin and Client. The admin is a user with special privileged access to all the files in the remote repository and access to all user accounts. The user is very similar to an admin in implementation, but with lower and more restricted privileges and only shared access to files.

   The following macros are used globally to keep track of the admin and the user privileges:

```
#define __USERS_MAX__         256
#define __USERNAME_ADMIN__    "admin"
#define __PASSWORD_ADMIN__    "12345"
#define __USERNAME_LEN_MAX__ 256
#define __PASSWORD_LEN_MAX__ 256
#define __FILES_OWNED_MAX__  256
#define __FILENAME_LEN_MAX__ 256
```

   Along with these, there is a **Role** enum which holds the roles and a **Record** struct which is written to the disk and is the schema of the user database.

   The admin can: create/remove user, create/share files, access all the files, view all accounts. The user can: change password, create/share files and access owned/shared files.

2. ### File Locking

   The implementation specifies two kinds of **pthread_mutex** advisory locks: one coarse–grained lock for the user database and many fine–grained locks for locking individual lines of files. These locks are set up and initialised when the shared memory of the application is loaded and initialised. At various stages of the implementation, such as when a user tries to edit a file, or when a user tries to flush a file to the disk, or when heartbeat directory synchronisation packets are exchanged, if there exists any operation involving modifying/looking up the database file, then the master lock is applied on the database file. Edit locks are constantly acquired and released when editing a file, based on the line currently being edited. An **ActiveLine** struct carries metadata and diff information about the line edits and the lock information. When a client closes a file or exits, then the server releases all the locks acquired by that user.

3. **Concurrency Control & Data Consistency**

   In this implementation, concurrency control is seen in two main areas: client–server interaction and client–client interaction. One of the interaction is between multiple clients, when all of them try to edit the same file. To prevent race conditions, the lines are locked per user using POSIX mutexes. Concurrent editing on different lines is allowed. Files are also flushed to the disk concurrently, which is triggered when any of the clients who has a file open triggers a manual flush action, which saves the file for every other client in possession of/with shared access to that file.

4. **INET Sockets**

   For the type, i.e., client–server interaction, the server runs an infinite loop , constsantly listening in at the port 8000 to detect any new clients. When a new (registered) client tries to log in to the application, the server establishes a connection by forking a new child process dedicated to that particular client and then passes the control to the client handler, which initialises/loads the appropriate data for that particular client account. The current implementation works for the localhost only.

5. **Inter–Process Communication**

   The implementation specifies four types of inter–process communication: sockets, shared memory, asynchronous signals and synchronous signals. The POSIX shared memory used in this implementation is a chunk of anonymous RAM allocated to be used between all the parent server and forked child client processes without having to pass around data of the users database, active files, lock information, etc.

   Asynchronous signals are used primarily for debugging and error handling. The implementation specifies a **`zombie_reaper`** signal handler to signal to the parent server dæmon to clean up any existing process table entries and reap the exit statuses of any terminated children. This handler handles the **`SIGCHLD`** signal sent asynchronously when any of the children die. The **`WNOHANG`** flag instructs the server not to go to sleep when waiting to reap zombie processes and only handle interrrupts when they arrive.

   Synchronous signals are custom–built wrappers around network packets passed between the server and clients, or amongst themselves. Every 50 milliseconds (20 frames per second), a heartbeat synchronisation request is sent from a client to the server, to get back an updated directory, which might have been modified by some other client. Every other signal is fired asynchronously by a client, but handled synchronly when the server checks in for pending signals. The response signals also function the exact same way in reverse. There is also a **`SIGPIPE`** handler to prevent any child terminating from breaking the pipes of other clients.

   The implementation specifies the following enumeration of synchronous signals (packets):

```c
typedef enum {

    PKT_LOGIN_REQ  , PKT_LOGIN_RES,   // login
    PKT_DIR_REQ    , PKT_DIR_RES,     // directory sync
    PKT_FOPEN_REQ  , PKT_FOPEN_RES,   // retrieve and open file
    PKT_EDIT_REQ   ,                  // broadcast diffs in current file
    PKT_SYNC_REQ   , PKT_SYNC_RES,    // receive edit broadcasts for file
    PKT_FLUSH_REQ  , PKT_FLUSH_RES,   // write file to disk
    PKT_USERS_REQ  , PKT_USERS_RES,   // retrieve users from database
    PKT_ADDUSER_REQ, PKT_ADDUSER_RES, // add user
    PKT_RMUSER_REQ , PKT_RMUSER_RES,  // remove user
    PKT_CHPSWD_REQ , PKT_CHPSWD_RES,  // change password
    PKT_MKFILE_REQ , PKT_MKFILE_RES,  // create file
    PKT_FLINK_REQ  , PKT_FLINK_RES,   // share file
    PKT_TERM,                         // error handling if client suddenly exits

} PacketType;
```

   Each packet is handled separately, depending on the role: admin or client, and only that information permitted is processed/extracted/modified. The naming convention has been taken from standard UNIX concepts. For example: **`FOPEN`** (from the library function **`fopen`**), **`FLUSH`** (from the library function **`fflush`**), **`FLINK`** (shared access is treated as soft linking, since multiple userspace files point to the same file on disk).

Challenges Faced, Solutions & Known Bugs

- **`Delayed Zombie Reaping`**: In earlier versions, the **`wait`** call was made inside the parent when a child was forked. This meant that zombies would be reaped only if a new client connected, and would fill up the process table. So, the wait was moved to an asynchronous signal (**`SIGCHLD`**) and its handler.

- **`Missing Packets`**: The MTU for my system is configured to approximately 1.5 KB. But the array holding file information is approximately 16 KB. So, created/shared files would go missing or sometimes cause broken pipes, because illegal/null information was being received. To fix this, I had to loop over until all the bytes arrived, using the **`MSG_WAITALL`** flag.

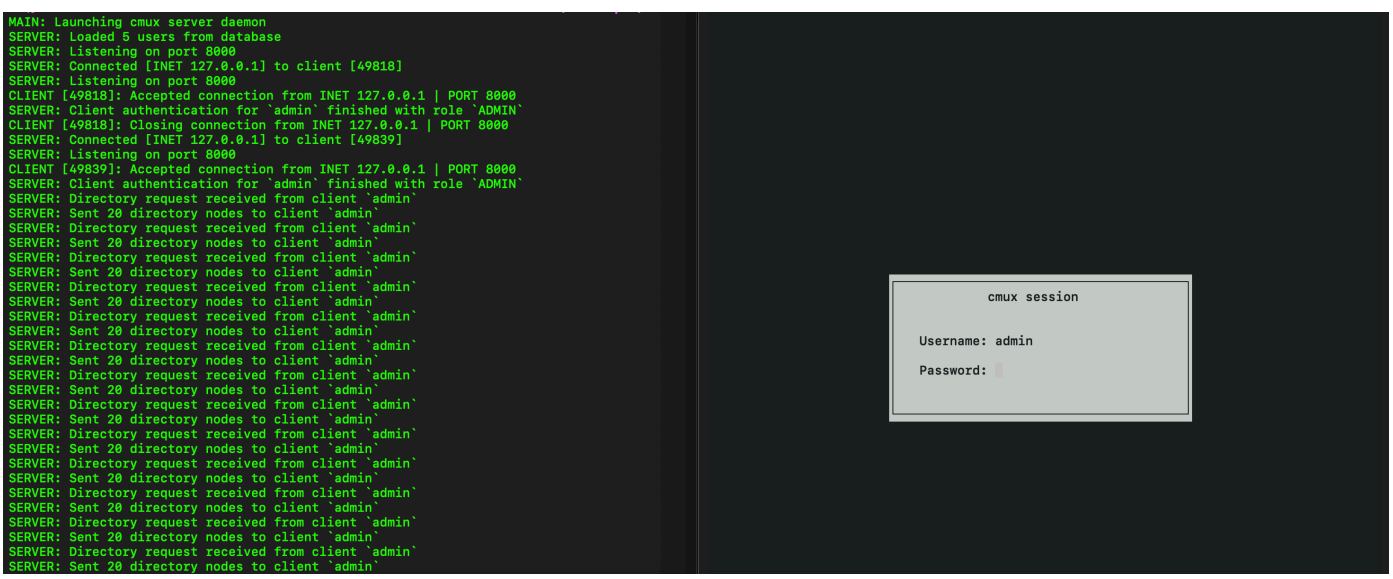
- **`Edit Deadlock`**: When a line is locked for typing by another client, and some other client tries to edit the line, the implementation jumps over the edit and prevents any race condition. But, because the client tried to acquire the mutex and got rejected, the server still thinks the client is waiting for the lock and once it is available, assigns it to that client. But, the lock information is stored in an array, independently of the client process itself, the client cannot come back and unlock the mutex, any other client trying to edit that line goes into deadlock and the UI freezes. To fix this, the mutex is unlocked just before skipping the edit, so that the server can update the array and release the lock.

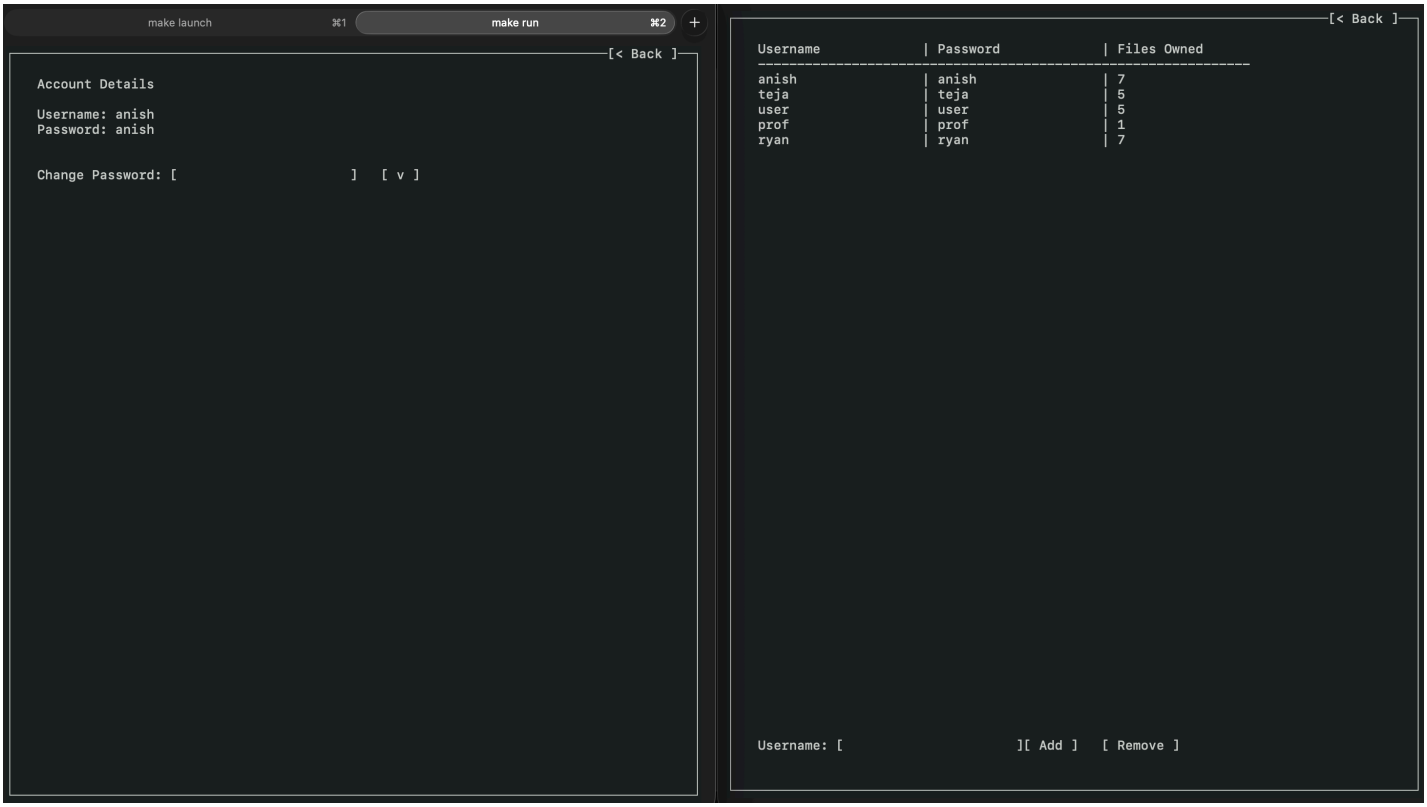
- **`Directory Sync Deadlock`**: In earlier versions, the client would log in and synchronously fetch the directory structure from the server once and would enter an edit mode. Any other directory fetching or file open opening requests would be ignored, since the server process would handle only one initial directory fetch and spin in its main loop. Specific parts of the UI would send requests, but would get ignored, which caused UI freezes and deadlock. To fix this, I had to move the packet handlers inside the loop so it would check for directory requests at all times. This later evolved to function as the heartbeat signal between the client and the server.

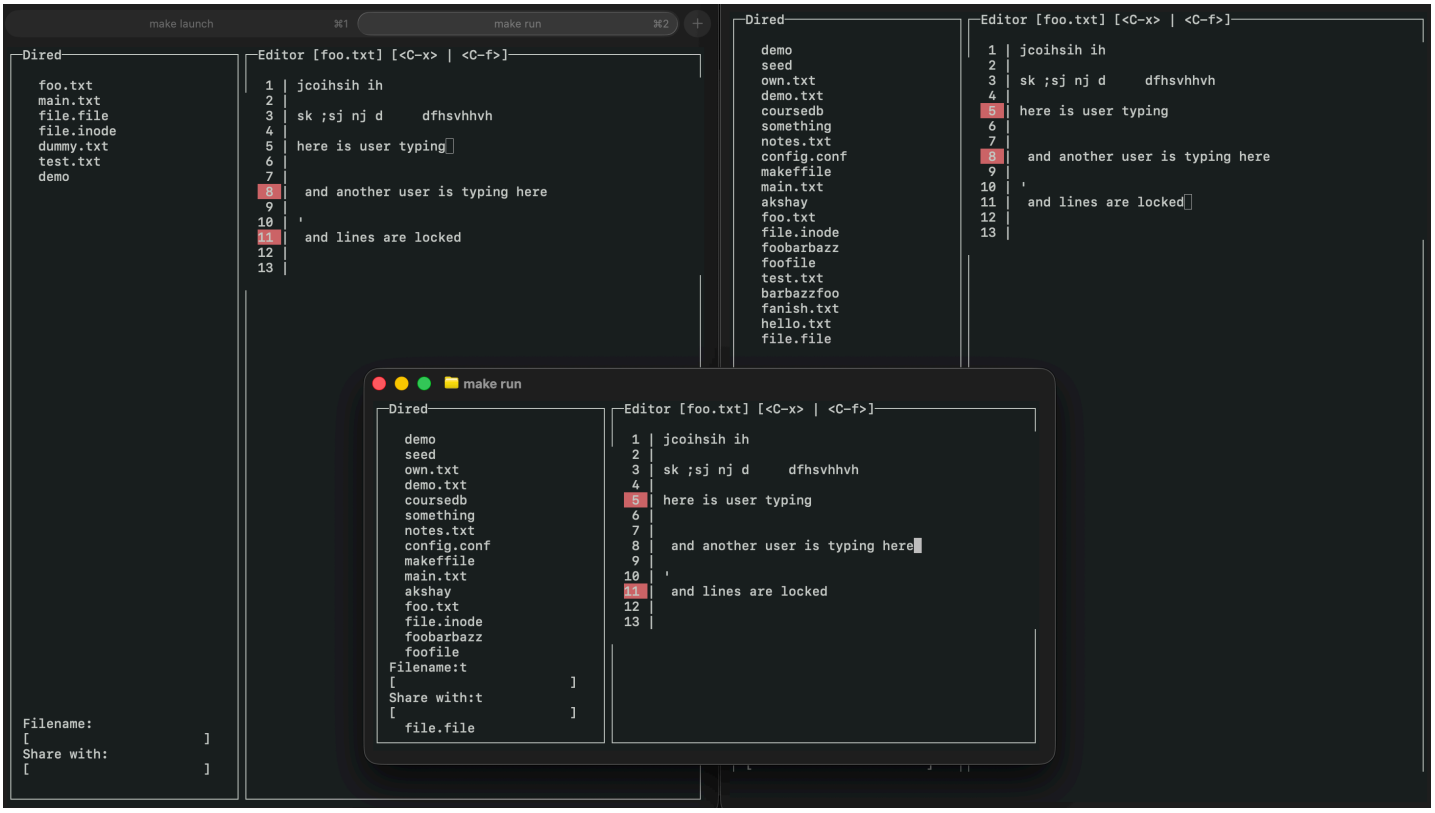
- **`UI Flickering`**: Due to the high refresh rate of the UI windows, placing the cursor somewhere undesirable or clicking somewhere unintended may cause slight blinking and/or flickering of the UI.

- **`Subdirectories`**: Currently, users cannot create subdirectories in the remote repository and doing so will cause parse errors and illegal pointer/undefined behaviour.

Sample Output




make launch ⌘1
make run ⌘2
[< Back]
Account Details
Username: anish
Password: anish
Change Password: [ ] [v]
[< Back]
Username | Password | Files Owned
anish | anish | 7
teja | teja | 5
user | user | 5
prof | prof | 1
ryan | ryan | 7
Username: [ ][Add] [Remove]


make launch ⌘1
make run ⌘2
Dired
foo.txt
main.txt
file.file
file.inode
dummy.txt
test.txt
demo
Filename:
[ ]
Share with:
[ ]
Editor [foo.txt] [<C-x> | <C-f>]
1 | jcoihsih ih
2 |
3 | sk ;sj nj d dfhsvhhvh
4 |
5 | here is user typing
6 |
7 |
8 | and another user is typing here
9 |
10 | '
11 | and lines are locked
12 |
13 |
make run
Dired
demo
seed
own.txt
demo.txt
coursedb
something
notes.txt
config.conf
makeffile
main.txt
akshay
foo.txt
file.inode
foobarbazz
foofile
Filename:t
[ ]
Share with:t
[ ]
file.file
Editor [foo.txt] [<C-x> | <C-f>]
1 | jcoihsih ih
2 |
3 | sk ;sj nj d dfhsvhhvh
4 |
5 | here is user typing
6 |
7 |
8 | and another user is typing here
9 |
10 | '
11 | and lines are locked
12 |
13 |
test.txt
barbazzfoo
fanish.txt
hello.txt
file.file